

Arrays

An array is a collection of items of the same variable type that are stored at contiguous memory locations. It is one of the most popular and simple data structures used in programming.

Initialization of arrays

```
arr = [2,3,45,23,1,3,5,7,8,65,34]
```

Types of arrays

1. Static arrays(fixed arrays)
 - This type of array cannot be altered or updated once it has been initialised
 - The size of the array needs to be defined on initialization
 - `arr = [0] * 5`
 - Can be very inefficient when you don't know how many elements to work with
2. Dynamic arrays
 - The size of the array changes according to the users requirements, The array can be altered and updated after initialization
 - The size does not need to be defined on initialization
 - `arr = []`

Common operations in arrays

1. Accessing an element (Use of indexes, usually integers, to access data) $O(1)$
2. Replacing values in an array $O(1)$
3. Searching $O(n)$
4. Inserting $O(n)$
5. Deleting $O(n)$

1. Accessing an element

- In other terms Traversal, is the process of accessing each element in the array sequentially, typically to perform a specific operation, such as searching, sorting, or modifying the elements.
- The most common methods of traversal include iterating through the array using loops like for, while, or foreach.

Types of array traversal

1. Linear traversal - Visiting each element in the array, starting from the first element, and during this each element is printed, one after the other.

Example:

```
arr = [1, 2, 3, 4, 5]
```

```
print("Linear Traversal: ", end=" ")
```

```
for i in arr:
```

```
    print(i, end=" ")
```

```
print()
```

2. Reverse Traversal - Visiting each element from the last element moving to the first element

Example

```
arr = [1, 2, 3, 4, 5]
```

```
print("Reverse Traversal: ", end="")
```

```
for i in range(len(arr) - 1, -1, -1):
```

```
    print(arr[i], end=" ")
```

```
print()
```

Methods of array traversal

1. For loop

- Allows you to iterate over an array by specifying an initial index, a condition and an update to the index after each iteration.

2. While loop

- The loop continues to execute as long as the condition specified is true.
- The loop stops based on the condition other than the size.

Example:

```
arr = [10, 20, 30, 40, 50]
```

```
n = len(arr)
```

```
i = 0
```

```
print("Traversal using while loop: ", end=" ")
```

```
while i < n:
```

```
    print(arr[i], end=" ")
```

```
    i += 1
```

```
print()
```

3. Foreach loop

- Simplifies array by directly accessing each element of the array without needing to manually manage the index.
- Used for simplicity and readability when there is need to access and process each item in the array

Example

```
arr = [10, 20, 30, 40, 50]
```

```
print("Traversal using foreach loop:", end=' ')
```

```
for value in arr:
```

```
    print(value, end=' ')
```

```
print()
```

2. Searching elements

- Searches an element by index of the value through linear search or binary search

Example:

```
arr = [10, 20, 30, 40, 50]
```

```
target = 30
```

```
found = False
```

```
# Linear search using traversal
```

```
for i in range(len(arr)):
```

```
    if arr[i] == target:
```

```
        found = True
```

```
        break
```

```
if found:
```

```
    print("Element found!")
```

```
else:
```

```
    print("Element not found!")
```

3. Inserting elements

- At the beginning
 - Insert an element at the beginning of the array
 - Use *Insert()*,

```
numbers = [2, 3, 4]
```

```
numbers.insert(0, 5) # insert 5 at the beginning
```

```
print(numbers)
```

- At a given position
 - Insert an element at a given position in the array
 - Use *Insert()*

```
numbers = [1, 2, 4, 5]
```

```
numbers.insert(2, 3) # insert 3 at index 2 (third position)
```

```
print(numbers)
```

- At the end
 - Use *append()*

```
numbers = [1, 2, 3]
numbers.append(4)    # adds 4 at the end
print(numbers)
```

4. Deleting Elements

- Deletion in arrays means removing an element from a specific position or based on a condition.
- When an element is deleted, other elements shift to fill the empty space, just like when someone leaves a line and everyone moves forward

Example: deleting at the beginning

```
numbers = [10, 20, 30, 40]
numbers.pop(0)    # or del numbers[0]
print(numbers)
```

Example of deletion at a given position

```
numbers = [10, 20, 30, 40, 50]
position = 2
del numbers[position] # removes element at index 2 (30)
print(numbers)
```

Example of deletion of first occurrence

```
numbers = [10, 20, 30, 20, 40]
numbers.remove(20) # removes the first 20 only
print(numbers)
```

Example of deletion of all occurrences of a value

```
numbers = [10, 20, 30, 20, 40, 20]
value_to_delete = 20
numbers = [x for x in numbers if x != value_to_delete]
print(numbers)
```

Example of deletion at the end

```
numbers = [10, 20, 30, 40]
numbers.pop() # removes the last element (40)
```

```
print(numbers)
```

Limitations of Arrays

- Fixed Size
- Inefficient when inserting and deleting

Hashmaps

- A hashmap (also called dictionary) is a data structure that stores data in key-value pairs
- It allows fast access, insertion and deletion using a unique key, instead of an index like in arrays
- Think of it like a labelled locker, each locker stores something and you can access it instantly if you know its label.

Initialization of hashmaps

```
Students_scores = {}
```

```
Students_scores = {1: 85, 2: 90, 3: 78, 4: 92}
```

Characteristics of hashmaps

- Stores data as key-value pairs
- Key must be unique
- Values can be duplicated
- Unordered - the order of the elements does not matter

Common operation in hashmaps

1. Accessing elements

- Accessing value using its key

```
scores = {1: 85, 2: 90, 3: 78, 4: 92}
```

```
print("Score of key 2:", scores[2])
```

2. Traversing a hashmap

- Can loop through keys, values or both

```
scores = {1: 85, 2: 90, 3: 78}
```

```
print("Keys:")  
for key in scores.keys():  
    print(key)
```

```
print("\nValues:")
for value in scores.values():
    print(value)

print("\nKey-Value pairs:")
for key, value in scores.items():
    print(key, ":", value)
```

3. Inserting elements

- Can add a new key-value pair by simply assigning it

```
scores = {1: 85, 2: 90}
scores[3] = 88
print(scores)
```

4. Updating elements

- If a key already exists, assigning a new value will update it.

```
scores = {1: 85, 2: 90, 3: 88}
scores[2] = 95
print(scores)
```

5. Searching for an element

- You can check if a key exists using the **in** keyword

```
scores = {1: 85, 2: 90, 3: 78}

if 2 in scores:
    print("Key found! Value =", scores[2])
else:
    print("Key not found")
```

6. Deleting elements

- Deleting by key

```
scores = {1: 85, 2: 90, 3: 78}
del scores[2]
print(scores)
```
- Delete using pop

```
scores = {1: 85, 2: 90, 3: 78}
removed = scores.pop(2)
print("Removed value:", removed)
print(scores)
```

- Delete all entries

```
scores = {1: 85, 2: 90, 3: 78}
scores.clear()
print(scores)
```

- HashMaps are based on **hashing**, where keys are converted into hash codes that decide where the data is stored in memory.
- Ideal for problems involving **fast lookups**, **counting**, and **mapping relationships**.
- Common interview problems:
 - Counting frequencies
 - Detecting duplicates
 - Two-sum problem (finding pairs that add up to a target)